

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 05 · LECTURE NOTES

Hash Functions

What they are for, and the three misuses.

MODULE OUTCOMES

Name the three security properties and distinguish them.

Choose a hash function for a stated purpose.

Explain length extension and why HMAC avoids it.

Name the three standard misuses.

Explain Merkle trees and content addressing.

1 The three properties

Property	What is infeasible	Security level for an n-bit output
Preimage resistance	Given $H(m)$, find any m producing it.	2^n — the full output length.
Second preimage resistance	Given m , find a different m' with the same hash.	2^n .
Collision resistance	Find <i>any</i> pair m, m' with the same hash — the adversary chooses both.	$2^{n/2}$ — the birthday bound, which is the number people get wrong.

Collision resistance is the weakest of the three to break and the one that matters most in practice. SHA-256 provides 128 bits of collision resistance, not 256 — because the adversary gets to choose both messages, and the birthday paradox (CSCE 658 Module 02) halves the exponent. **This is exactly why output lengths are specified at double the intended security level**, and why a 128-bit hash offers only 64 bits against collisions, which is now reachable.

COLLISION RESISTANCE IS WHAT SIGNATURES DEPEND ON, AND IT IS THE FIRST TO FALL

A signature is computed over a hash of the message rather than over the message itself (Module 07) — for both performance and structural reasons — so two messages with the same hash have exactly the same valid signature.

Which makes a collision directly exploitable: obtain a signature on a benign document, then present the colliding malicious document bearing that same signature — and both verify correctly, because verification only ever sees the hash.

And this was never hypothetical. MD5 collisions were used to produce a fraudulent certificate authority certificate that chained to a real root, and SHA-1 collisions were later demonstrated for real document formats where the colliding pair rendered as two different documents.

So the practical rule is stated by purpose: MD5 and SHA-1 are broken for anything requiring collision resistance, which includes every signature and every commitment — while remaining perfectly adequate for non-adversarial integrity checks like detecting a corrupted download, where nobody is constructing the input.

2 Choosing

GENERAL CRYPTOGRAPHIC USE

SHA-256 / SHA-512	the safe default
SHA-3 / SHAKE	different internal structure; no length extension
BLAKE2 / BLAKE3	faster, keyed mode built in, no length extension

PASSWORDS -- NONE OF THE ABOVE

Argon2id, scrypt, bcrypt. A fast hash is exactly wrong here, and Module 09 explains why at length.

NON-ADVERSARIAL CHECKSUMS

CRC32, xxHash. Fast, not secure, and that is entirely fine when nobody is attacking the input.

BROKEN FOR COLLISION RESISTANCE

MD5, SHA-1. Still acceptable as non-adversarial checksums; never in a signature or a commitment.

The password row is the one that matters most. Using SHA-256 for password storage is a defect rather than a weaker choice — the property you need there is slowness, which every hash in the first group is specifically designed not to have (Module 09 §1). And separating 'broken' from 'broken for what' is the genuinely useful distinction: it lets you assess an inherited system accurately rather than flagging every MD5 as critical.

3 Length extension

GIVEN $H(\text{SECRET} \parallel M)$ YOU CAN COMPUTE $H(\text{SECRET} \parallel M \parallel \text{PAD} \parallel M')$ WITHOUT KNOWING THE SECRET

SHA-256 and SHA-1 process the message in fixed-size blocks, and the final output is the internal state — the Merkle–Damgård construction — so knowing the hash means knowing the state, and knowing the state means being able to continue hashing from it.

Which breaks the naive MAC completely. A system authenticating messages with $H(\text{key} \parallel \text{message})$ can have arbitrary data appended by an adversary who never learns the key, and the resulting tag is valid — the adversary needs only the original hash, the original message length, and the data to append.

This has produced real vulnerabilities, particularly in signed-URL and signed-cookie schemes built from a bare hash rather than from a MAC — where appending `&admin=true` to an authenticated query string was exactly the attack.

HMAC's nested structure exists precisely to prevent it (see the equation) — and SHA-3, BLAKE2, and BLAKE3 are not susceptible at all, because their constructions do not expose the full internal state as output. Which is a reason to prefer them where you have a free choice.

$$\text{HMAC}(k, m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$$

- **ipad, opad** — two distinct fixed padding constants, which make the inner and outer keys different.
- **The inner hash** — produces a fixed-length digest, so **the outer hash's input has a fixed length that an adversary cannot extend**.
- **The outer hash** — re-keys over that digest, so **the output is not a continuable internal state for the message**, which is exactly what removes the extension property.
- **The result** — **length extension is impossible, and the security proof requires only modest assumptions about H** — notably, HMAC-MD5 remained unbroken as a MAC well after MD5's collision resistance fell, because a MAC does not depend on collision resistance. **The nesting is not arbitrary**, and explaining it is what makes HMAC memorable rather than mysterious.

4 Misuses, and good uses

- **A hash used as a MAC. It is unkeyed, so anybody including the adversary can recompute it** — it detects accidental corruption and nothing adversarial at all (Module 04 §1).
- **A fast hash used for passwords. Speed is precisely the attacker's advantage**, and billions of guesses per second on commodity hardware follow directly (Module 09).
- **A hash used to hide a low-entropy value. Hashing an email address, a phone number, a date of birth, or a national identifier conceals essentially nothing** — the input space is small enough to enumerate exhaustively, so the hash is a reversible encoding in practice.
- **And a fourth worth naming: truncating a hash without accounting for the birthday bound**, which halves the collision security again — a 64-bit truncation offers 32 bits, which is trivially reachable.

- **The third misuse is the one that appears in privacy claims — 'we only store hashed email addresses' is not de-identification**, and it is routinely presented to users and regulators as though it were. **This is the most consequential misuse outside engineering**, and a salted, slow construction is the minimum that changes the answer.

AND THE CONSTRUCTIONS WORTH KNOWING

Content addressing. Name the data by its hash — and the name then verifies the content, so retrieval from an untrusted source is self-checking. This is how Git, container registries, and package lockfiles obtain their integrity (CSCE 701 Module 08 §3).

Merkle trees. Hash pairs of hashes upward to a single root; the root commits to every leaf beneath it, and a single branch of the tree proves membership using only $\log n$ hashes rather than the whole structure.

Which gives efficient verification of a small part of a very large structure — the basis of certificate transparency logs (Module 07 §4), distributed version control, block-chained ledgers, and verified file systems.

And commitment: publish $H(\text{value})$ now, reveal the value later, and you cannot have changed it in between — binding and hiding from one primitive. **Though this requires the value to carry enough entropy, or to be accompanied by a random salt**, which is the third misuse arriving again in a different guise.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aumasson — Serious Cryptography, chapters 6 and 7	The whole module , with the construction internals and the practical choice guidance.
Boneh & Shoup, chapters 8 and 14 (free)	§1's definitions and §3's HMAC proof — including why a MAC does not require collision resistance.
The SHAttered SHA-1 collision (free)	§1's demonstration , with the colliding PDF pair and the cost analysis.
Certificate Transparency (free)	§4's Merkle trees in production — and the infrastructure Module 07 §4 depends on.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State which of the three properties** each of five hash uses in your own code actually requires.
- 2 **Compute the collision security** of a 256-bit hash truncated to 128 bits, and to 64.
- 3 **Explain why a signature forgery follows from a collision**, step by step.
- 4 **Find an MD5 use you can safely leave alone** and justify it.
- 5 **Demonstrate length extension** against $H(\text{secret} \parallel \text{message})$ using a library that exposes the state.
- 6 **Show that HMAC resists the same attempt.**
- 7 **Explain the nesting** in your own words.
- 8 **Enumerate a hashed low-entropy space** — hash 10,000 plausible phone numbers and match them.
- 9 **Build a Merkle tree** over 1000 items and produce a membership proof.
- 10 **Verify the proof without the other 999 items.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three properties and the security level of each.
- 2 Why is collision resistance only $n/2$ bits?
- 3 Why does a collision break a signature scheme?
- 4 What does 'broken' mean for MD5, and what remains acceptable?
- 5 Which hash for general use, for passwords, and for checksums?
- 6 State the length extension property and why it holds.
- 7 What real vulnerability class did it produce?
- 8 Explain HMAC's nesting and what it achieves.
- 9 Name the three misuses, and the fourth.
- 10 Explain content addressing, Merkle trees, and commitment.